SLOT PENGUMPULAN WORKBOOK PRAKTIKUM 01 • FLUTTER

WORKBOOK FLUTTER ANDROID

StudyLog — Aplikasi Catatan Perkuliahan

State & Event • Multi-screen Navigation • Local SQLite • Project-Based Learning

Diajukan untuk Memenuhi Tugas Praktikum Mata Kuliah

PEMROGRAMAN APLIKASI BERGERAK

Dosen Pengampu: Pramudya Prima Insan, S.Kom., M.Kom. | Kelas: IFB5J

DISUSUN OLEH ANGGOTA KELOMPOK:

No	Nama Lengkap Mahasiswa	NIM	Peran & Kontribusi
1	**Ryanda Hermawan**	2411068	Arsitektur SQLite 3 Tabel Relasional & Git
2	**Dewi Nofitta Sari**	2411065	Antarmuka Multi-Layar, Form & State Provider
3	**Bintang Sahda Rayyu Sonora**	2411025	Pengujian Sistem, Video Demo & Dokumentasi

Repository GitHub	https://github.com/rillsussy/studylog
Tautan Video Demo	https://youtu.be/TQ_v1NJ2w4k

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MULIA

2026

PANDUAN PRAKTIKUM / PROYEK BERKELANJUTAN

WORKBOOK FLUTTER ANDROID

StudyLog — Aplikasi Catatan Perkuliahan

State & Event • Multi-screen Navigation • Local SQLite • Project-Based Learning

Mata kuliah Pemrograman Aplikasi Bergerak

Program studi: Informatika

Institusi Universitas Mulia

Edisi Pembelajaran Terpadu 2026

Petunjuk Penggunaan Workbook

Workbook ini adalah panduan operasional bagi mahasiswa untuk mengerjakan praktikum Android menggunakan Flutter (bahasa Dart). Ia disusun berdasarkan tiga materi asal: State, Events & User Interaction; Navigation, Lifecycle & ViewModel; dan Local Persistence & Room Database. Konsep inti dipertahankan, sedangkan teknologi implementasi disesuaikan ke Flutter dan SQLite.

Modul Mobile Programming terlampir digunakan sebagai acuan alur mengajar Flutter: pengenalan widget, daftar dan detail data, formulir, navigasi, serta pola pengembangan CRUD. Contoh "Data Poli" dari modul dialihkan menjadi proyek "StudyLog" agar tiga materi disatukan dalam satu aplikasi. Bagian API/mockapi dari modul asli bukan cakupan wajib unit ini.

YANG HARUS DISIAPKAN Perangkat Windows/macOS/Linux yang menjalankan Flutter SDK dan Android SDK, editor Android Studio/VS Code, emulator Android atau ponsel Android dengan USB debugging, dan Git. Versi aktual diverifikasi lewat flutter doctor.

Identitas Mahasiswa dan Kelompok

Isian	Diisi mahasiswa
Nama / NIM	1. Ryanda Hermawan (2411068) 2. Dewi Nofitta Sari (2411065) 3. Bintang Sahda Rayyu Sonora (2411025)
Kelas / Kelompok	IFB5J — Kelompok StudyLog
Anggota tim / peran	• Ryanda Hermawan: Arsitektur SQLite 3 Tabel Relasional, Migrasi Skema & Git • Dewi Nofitta Sari: Desain UI Multi-Screen, Form Validasi & State Provider • Bintang Sahda Rayyu Sonora: Pengujian Sistem (Unit/Widget/HP), Video Demo & Dokumentasi
Repository GitHub	https://github.com/rillsussy/studylog | Video Demo: https://youtu.be/TQ_v1NJ2w4k
Perangkat uji / Android	Ponsel Android Fisik (USB Debugging & APK Debug Android)

Peta Isi

Bab	Fokus	Hasil
1. Orientasi proyek	Tujuan, kebutuhan, rencana PjBL	Backlog dan desain awal
2. Persiapan alat	Flutter SDK, emulator, struktur proyek	Aplikasi template berjalan
3. Praktikum 1	State, event, validasi	Form interaktif
4. Praktikum 2	Navigator, lifecycle, Provider	Multi-screen data sementara
5. Praktikum 3	SQLite, model, repository, CRUD	Data tersimpan permanen
6. Proyek mandiri	Perluasan fitur, pengujian, Git	Produk akhir dan demo
7. Instrumen evaluasi	Checklist, rubrik, refleksi	Bukti capaian belajar

1. Orientasi dan Capaian Pembelajaran

1.1 Skenario

Unit proyek: StudyLog, aplikasi Android untuk mencatat materi kuliah tanpa ketergantungan internet. Pengguna membuka beranda, melihat daftar catatan, memasukkan catatan baru, melihat detail, menyunting isinya, lalu menghapus jika tidak diperlukan.

1.2 Capaian terukur

- Mahasiswa menjelaskan keterkaitan state, event, validasi, dan perubahan tampilan Flutter serta membuktikannya dalam program.
- Mahasiswa mengembangkan navigasi beranda → daftar → detail → form dengan tombol kembali yang benar.
- Mahasiswa memisahkan state UI dari tampilan menggunakan ChangeNotifier dan Provider serta memahami batasnya terhadap penyimpanan permanen.
- Mahasiswa merancang tabel catatan dan menjalankan operasi create, read, update, delete pada SQLite lokal.
- Mahasiswa menyusun dokumentasi pengujian, rekam jejak Git, dan demonstrasi hasil proyek serta mempertanggungjawabkan kontribusi individual.

1.3 Peta kompetensi

Checkpoint	Kompetensi	Kondisi data	Output
M1	UI interaktif + form validasi	RAM pada satu layar	Screenshot form dan pesan
M2	Multi-screen + pengelolaan state	RAM lintas layar	Demo navigasi & rotasi
M3	CRUD + database SQLite	Persisten di perangkat	Video CRUD dan uji buka ulang

BATASAN Satu tabel catatan digunakan untuk memastikan seluruh mahasiswa mampu menyelesaikan kompetensi dasar. Untuk proyek semester dengan ketentuan tiga tabel, perlu pengembangan skema relasional terpisah.

1.4 Alokasi belajar (usulan)

Sesi	Aktivitas studio/praktikum	Asesmen formatif
Sesi 1 — 150 menit	Setup, widget, input, Form dan setState	Demo input valid/tidak valid
Sesi 2 — 150 menit	Navigator, halaman, Provider, lifecycle	Demo alur kembali dan data sementara
Sesi 3 — 150 menit	Model, SQL, repository, CRUD, pengujian	Demo data bertahan setelah restart
Sesi 4 — 150 menit	Pengembangan mandiri, debugging, presentasi	Produk akhir dan refleksi

2. Perangkat, Persiapan, dan Alur Kerja

2.1 Prasyarat

☑ Flutter SDK versi stabil sudah terpasang; lokasi SDK dikenali terminal.

- ☑ Android Studio / VS Code tersedia, plugin Flutter & Dart aktif.
- ☑ Android SDK & Android emulator atau HP Android tersambung.
- ☑ Git terpasang untuk pencatatan progres.
- ☑ Folder praktikum berisi milestone_01_form, milestone_02_navigation, dan milestone_03_sqlite.

2.2 Perintah pemeriksaan

```
flutter --version
flutter doctor
flutter devices
```

Jika flutter doctor melaporkan masalah Android SDK, selesaikan komponen yang diberi tanda sebelum menulis kode. Jangan menjadikan emulator web sebagai pengganti uji database sqflite pada Android.

2.3 Buat scaffold proyek Android

```
flutter create studylog_flutter
cd studylog_flutter
flutter run
```

Setelah aplikasi template berhasil dibuka di emulator/perangkat, salin file pubspec.yaml dan lib/main.dart dari folder checkpoint pertama. Jangan menyalin folder android/ lama atau hasil build antartahap. Jalankan:

```
flutter pub get
flutter analyze
flutter run
```

2.4 Aturan checkpoint Git

```
git init
git add .
git commit -m "M1: formulir interaktif"
# setelah tahap 2 dan 3, commit ulang dengan pesan sesuai tahap
```

Pada folder yang sama, lanjutkan M2 dengan mengganti file checkpoint. Lakukan commit sebelum penggantian agar seluruh perjalanan kode tersimpan. Gunakan .gitignore dari hasil flutter create; jangan memasukkan folder build/ ke repository.

3. Praktikum 1 — Form Interaktif

Tujuan, waktu, dan produk

Tujuan: membuat aplikasi Flutter satu layar yang dapat menerima data, mengecek kelengkapan, merespons tombol, dan memberi umpan balik. Alokasi 150 menit. Produk: form mata kuliah, judul, isi, dengan hasil input yang muncul di layar; belum ada database.

3.1 Apa yang akan dipelajari

Konsep	Peran pada aplikasi
StatefulWidget	Layar yang memiliki data atau kondisi yang berubah
TextEditingController	Mengambil isi field dan melepaskan controller saat dispose
Form + GlobalKey<FormState>	Mengelompokkan validasi beberapa TextFormField
validator	Menghasilkan pesan jika nilai tidak memenuhi aturan
setState()	Memberi tahu Flutter bahwa data lokal berubah dan UI perlu dibangun ulang
SnackBar	Menyajikan pesan hasil aksi pengguna

3.2 Langkah kerja

- Buka proyek yang dibuat pada Bab 2. Pastikan emulator/HP terdeteksi pada menu target perangkat.
- Salin pubspec.yaml dan lib/main.dart dari milestone_01_form, kemudian jalankan flutter pub get.
- Amati fungsi main() yang memanggil StudyLogApp dan mengatur tema MaterialApp.
- Cari class CatatanFormSederhana dan identifikasi State, tiga TextEditingController, serta variabel _hasil.
- Temukan widget Form, masing-masing validator, dan tombol FilledButton.
- Jalankan aplikasi dan isi tiga field valid; tekan "Tampilkan Catatan"; amati Card dan SnackBar.
- Kosongkan satu field dan tekan tombol; catat perbedaan responsnya.
- Lakukan Hot Reload setelah mengubah teks judul layar dan amati hasil.

3.3 Potongan kode utama: validasi

```
final _formKey = GlobalKey<FormState>();
final _judul = TextEditingController();

TextFormField(
  controller: _judul,
  validator: (nilai) => (nilai == null || nilai.trim().isEmpty)
```

```
      ? 'Judul wajib diisi' : null,
)

void _simpanSementara() {
  if (!_formKey.currentState!.validate()) return;
  setState(() { _hasil = _judul.text.trim(); });
}
```

Penjelasan: event klik menjalankan fungsi _simpanSementara. Validator memeriksa input; setState hanya dipanggil jika valid. Pada tahap ini data dapat hilang ketika aplikasi dihentikan sepenuhnya karena belum disimpan pada penyimpanan lokal.

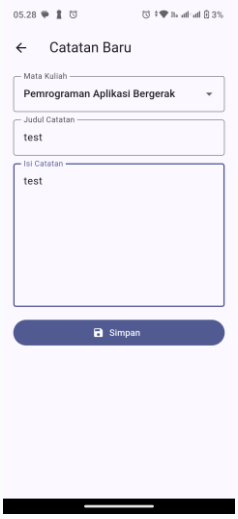
3.4 Uji praktik

ID	Skenario	Hasil yang diharapkan	Status
M1-T1	Semua field kosong, klik tombol	Pesan wajib diisi tampil	☑ Lulus ☐ Gagal
M1-T2	Satu field berisi spasi	Validasi menolak input	☑ Lulus ☐ Gagal
M1-T3	Semua field terisi	Card dan SnackBar muncul	☑ Lulus ☐ Gagal
M1-T4	Ubah judul, simpan lagi	Isi Card diperbarui	☑ Lulus ☐ Gagal

RUANG DOKUMENTASI | M1: Form Valid dan Kondisi Error Validasi



Kondisi Error: Field Kosong / Spasi Ditolak



Kondisi Valid: Input Siap Disimpan

3.5 Latihan mandiri

- Tambahkan field "Topik" dan buat validasi minimal tiga karakter.
- Ubah warna/ikon tombol ketika berhasil menyimpan.
- Jelaskan mengapa setState tidak sama dengan database dan bukan penyimpanan permanen.

- ☑ Form dapat menerima input.
- ☑ Validasi berjalan untuk data kosong.
- ☑ Hasil ditampilkan di UI.
- ☑ Perubahan disimpan dalam commit M1.

4. Praktikum 2 — Navigasi, Lifecycle, dan State

Tujuan: membangun empat halaman: Beranda, Daftar Catatan, Detail Catatan, dan Form Catatan. Data dikelola menggunakan ChangeNotifier/Provider pada memori aplikasi. Alokasi 150 menit.

4.1 Alur layar

Beranda → Daftar → Detail; Daftar → Tambah Form; Form → kembali ke Daftar setelah data ditambahkan. Tombol kembali harus mengembalikan pengguna ke halaman sebelumnya, bukan selalu ke Beranda.

4.2 Memasang dependensi

Salin pubspec.yaml serta lib/main.dart dari milestone_02_navigation ke proyek latihan yang sama. Jika ingin memasang langsung melalui perintah, gunakan:

```
flutter pub add provider
flutter pub get
flutter run
```

4.3 Kode navigasi dasar

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const DaftarPage()),
);
// Navigasi kembali dari form:
Navigator.pop(context);
```

push menambahkan halaman ke back stack dan pop menutup halaman teratas. Di Flutter, contoh ini memakai Navigator berbasis MaterialPageRoute, bukan NavController milik Jetpack Compose.

4.4 Kode state lintas halaman

```
class CatatanStore extends ChangeNotifier {
  final List<CatatanMini> _data = [];
  List<CatatanMini> get data => List.unmodifiable(_data);

  void tambah(CatatanMini catatan) {
    _data.add(catatan);
    notifyListeners();
```

```
  }
}
```

```
runApp(ChangeNotifierProvider(
  create: (_) => CatatanStore(),
  child: const StudyLogApp(),
));
```

Fungsi notifyListeners memberi sinyal agar Consumer pada halaman daftar membaca state terbaru. Data masih hanya berada di RAM; pada restart penuh, daftar kembali kosong.

4.5 Langkah kerja

- Jalankan checkpoint M2 dan pastikan beranda tampil.
- Masuk Daftar Catatan; amati empty state sebelum ada catatan.
- Tekan tombol +; isi tiga field lalu tekan "Simpan Sementara".
- Pastikan halaman form tertutup dan catatan langsung muncul di daftar.
- Ketuk salah satu item untuk membuka halaman detail.
- Tekan tombol kembali dari detail, lalu dari daftar; pastikan riwayat halaman sesuai.
- Lakukan Hot Reload, rotasi perangkat, dan force stop/restart; bandingkan kondisi data di setiap skenario.
- Catat keterbatasan state sementara untuk kebutuhan aplikasi pengguna nyata.

4.6 Observasi lifecycle

Kejadian	Prediksi mahasiswa	Hasil aktual
Navigasi Beranda → Daftar → Detail	Navigator.push menumpuk halaman pada back stack; tombol kembali (pop) mengembalikan ke Daftar lalu Beranda.	Sesuai prediksi. Urutan kembali konsisten (Detail → Daftar → Beranda), tidak melompat langsung ke awal.
Rotasi layar / rebuild widget	Widget dibangun ulang menyesuaikan orientasi layar, namun objek ChangeNotifier di Provider tetap hidup.	Sesuai prediksi. Daftar catatan tetap utuh tampil di layar setelah rotasi maupun rebuild widget.
Hot Reload	Kode UI diperbarui ke Dart VM tanpa menjalankan ulang main(), sehingga state di memori tetap terjaga.	Sesuai prediksi. Perubahan tampilan langsung terlihat cepat dan catatan selama sesi debug tidak hilang.
Force stop lalu buka ulang aplikasi	Pada M2 (RAM), isi memori terhapus saat proses mati; pada M3 (SQLite), data dibaca ulang dari studylog.db.	Sesuai prediksi. Di M2 daftar kosong kembali; di M3 seluruh catatan tetap tersedia setelah force stop & buka ulang.

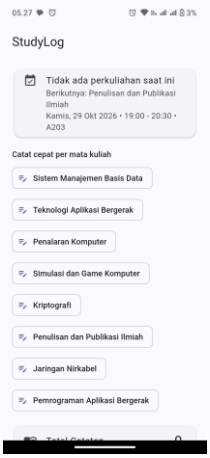
Catatan penting: ChangeNotifier membantu pengelolaan state ketika widget dibangun ulang selama objeknya masih hidup; ia tidak menjamin pemulihan state sesudah proses aplikasi

dimatikan. Persistensi membutuhkan mekanisme penyimpanan, yang dipelajari pada Praktikum 3.

4.7 Tugas pengayaan

- Tambahkan halaman "Tentang Aplikasi" dan hubungkan dari Beranda.
- Tampilkan jumlah total catatan pada halaman Daftar menggunakan Consumer.
- Refleksikan alasan data hilang sesudah aplikasi dihentikan sepenuhnya.

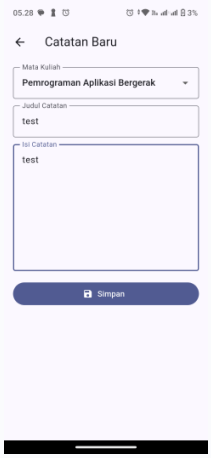
RUANG DOKUMENTASI | M2: Alur Empat Layar (Beranda, Daftar, Form, dan Detail)



1. Beranda & Jadwal



2. Daftar Catatan



3. Form Catatan



4. Detail Catatan

- ☑ Minimal empat layar dapat dibuka.
- ☑ Navigator push/pop bekerja benar.
- ☑ Data diperbarui via ChangeNotifier/Provider.
- ☑ Hasil rotasi dan restart dicatat.
- ☑ Commit Git M2 berhasil.

5. Praktikum 3 — SQLite, Repository, dan CRUD

Tujuan: mengubah aplikasi memori menjadi aplikasi yang benar-benar menyimpan data lokal Android. Alokasi 150 menit. Data yang telah disimpan harus tersedia saat aplikasi ditutup dan dibuka kembali.

5.1 Struktur tabel

Kolom	Tipe SQLite	Aturan
id	INTEGER	PRIMARY KEY AUTOINCREMENT
mata_kuliah	TEXT	NOT NULL
judul	TEXT	NOT NULL
isi	TEXT	NOT NULL

tanggal	TEXT	NOT NULL; format ISO 8601

5.2 Aliran tanggung jawab

UI → ViewModel (ChangeNotifier) → Repository → DatabaseHelper (sqflite) → SQLite. Saat pengguna mengubah catatan, UI meminta aksi ke ViewModel; repository menjalankan query; ViewModel memuat ulang daftar dan memberitahukan perubahan kepada UI.

5.3 Tambahkan paket

```
flutter pub add provider sqflite path
flutter pub get
```

PERHATIAN Pengujian SQLite pada workbook ditujukan untuk emulator/perangkat Android. Jangan memilih target Chrome ketika mendemonstrasikan penyimpanan database sqflite.

5.4 Struktur folder checkpoint final

```
lib/
  main.dart
  models/catatan.dart
  data/database_helper.dart
  data/catatan_repository.dart
  viewmodels/catatan_view_model.dart
  screens/beranda_page.dart
  screens/daftar_catatan_page.dart
  screens/detail_catatan_page.dart
  screens/form_catatan_page.dart
test/catatan_model_test.dart
```

5.5 Membuat model dan serialisasi

```
class Catatan {
  final int? id;
  final String mataKuliah, judul, isi;
  final DateTime tanggal;

  const Catatan({this.id, required this.mataKuliah,
    required this.judul, required this.isi, required this.tanggal});

  Map<String, Object?> toMap() => {
    if (id != null) 'id': id,
    'mata_kuliah': mataKuliah, 'judul': judul,
    'isi': isi, 'tanggal': tanggal.toIso8601String(),
  };
```

```
}
```

Lihat file lengkap lib/models/catatan.dart pada paket. Fungsi fromMap() diperlukan untuk mengonversi baris SQLite menjadi objek Dart ketika membaca daftar.

5.6 Membuat tabel melalui DatabaseHelper

```
final lokasi = join(await getDatabasesPath(), 'studylog.db');
final db = await openDatabase(lokasi, version: 1,
  onCreate: (db, version) async {
    await db.execute("""CREATE TABLE catatan (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      mata_kuliah TEXT NOT NULL, judul TEXT NOT NULL,
      isi TEXT NOT NULL, tanggal TEXT NOT NULL
    )""");
  },
);
```

Schema awal dibuat satu kali pada pembuatan database. Jika mahasiswa mengubah struktur tabel setelah aplikasi terpasang, diperlukan strategi migration/version yang sesuai—tidak cukup hanya mengedit perintah CREATE TABLE.

5.7 Membuat Repository

```
Future<List<Catatan>> semua() async {
  final db = await helper.database;
  final baris = await db.query('catatan',
      orderBy: 'tanggal DESC, id DESC');
  return baris.map(Catatan.fromMap).toList();
}

Future<int> hapus(int id) async {
  final db = await helper.database;
  return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
}
```

Metode tambah() dan ubah() pada file repository menggunakan db.insert dan db.update. Penggunaan whereArgs memastikan nilai parameter tidak dirangkai langsung ke string query.

5.8 ViewModel dan UI

```
Future<void> tambahCatatan(Catatan baru) async {
  await repository.tambah(baru);
  await muatCatatan();
}

Future<void> muatCatatan() async {
  isLoading = true; notifyListeners();
  try {
    _catatan = await repository.semua();
```

```
  } finally {
    isLoading = false; notifyListeners();
  }
}
```

Potongan kode ini disederhanakan untuk fokus alur; implementasi lengkap di paket menangani variabel error untuk kegagalan membaca database.

5.9 Langkah kerja utama

- Lakukan commit M2 terlebih dahulu. Salin seluruh folder lib dan test serta pubspec.yaml dari milestone_03_sqlite.
- Jalankan flutter pub get kemudian flutter analyze; periksa kesalahan impor atau nama file jika ada.
- Jalankan aplikasi pada emulator/HP Android dan buka Daftar Catatan.
- Tekan Tambah; isi mata kuliah, judul, dan isi; tekan Simpan.
- Pastikan catatan baru muncul. Buka detailnya, pilih Ubah, edit judul, lalu simpan.
- Buka detail dan tekan Hapus; pada dialog pilih Batal lalu verifikasi data masih ada.
- Ulangi Hapus dan tekan Hapus pada dialog; pastikan hanya item terpilih yang hilang.
- Tambahkan dua catatan baru. Tutup sepenuhnya aplikasi, lalu buka ulang. Pastikan kedua catatan tetap ada.
- Jalankan flutter test untuk menguji konversi model, kemudian simpan screenshot hasil run dan output terminal.

5.10 Matriks pengujian CRUD

Kasus	Langkah uji	Ekspektasi	Hasil
T1 — Create	Input 3 field, simpan	Item baru muncul	☑ Lulus
T2 — Read	Buka daftar dan detail	Data sesuai input	☑ Lulus
T3 — Update	Ubah judul, simpan	Data terbaru tampil	☑ Lulus
T4 — Delete batal	Hapus lalu Batal	Item tidak dihapus	☑ Lulus
T5 — Delete setuju	Hapus lalu konfirmasi	Item hilang	☑ Lulus
T6 — Persistence	Tutup aplikasi & buka lagi	Data masih tersedia	☑ Lulus
T7 — Validasi	Simpan field kosong	Pesan validasi tampil	☑ Lulus

5.11 Perintah pemeriksaan

```
flutter analyze
flutter test
flutter run
```

flutter analyze dan flutter test harus dijalankan mahasiswa pada lingkungan Flutter yang terpasang. Keberhasilan penyusunan dokumen atau keberadaan file kode saja bukan bukti bahwa aplikasi telah berhasil dikompilasi.

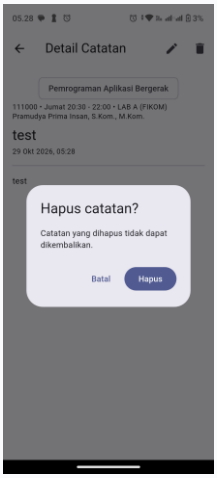
RUANG DOKUMENTASI | M3: Daftar SQLite, Detail Relasi, Dialog Hapus, & Uji Restart



Daftar, Cari & Filter SQLite



Detail & Metadata Relasi MK



Dialog Konfirmasi Batal/Hapus

Bukti Kompilasi, Pengujian Otomatis & Uji Persistensi Restart:
• flutter analyze: No issues found! | flutter test: 18/18 tests passed (model, CRUD FFI, migrasi v1→v3, widget)
• Video Demo CRUD & Tutup Paksa / Buka Ulang Aplikasi: https://youtu.be/TQ_v1NJ2w4k

- ☑ Database berhasil dibuat dan dibuka pada Android.
- ☑ CRUD tambah/tampil/ubah/hapus berhasil.
- ☑ Dialog konfirmasi hapus bekerja.
- ☑ Data tersedia setelah aplikasi dibuka ulang.
- ☑ Ada screenshot hasil pengujian dan commit M3.

6. PjBL — Pengembangan Mandiri StudyLog

6.1 Tantangan proyek

Tantangan: "Bagaimana membantu mahasiswa mengarsipkan catatan perkuliahan yang mudah dicari, diubah, dan digunakan meskipun koneksi internet tidak tersedia?" Produk minimal mengikuti checkpoint final. Setiap kelompok memilih setidaknya dua pengembangan fitur yang mempunyai kegunaan jelas bagi pengguna.

6.2 Pengembangan pilihan

Fitur pengembangan	Indikator keberhasilan	Tingkat
Cari catatan	Daftar terfilter berdasarkan kata kunci judul	Dasar
Urutkan catatan	Urutan terbaru/terlama dapat dipilih	Dasar

Kategori/mata kuliah	Pengelompokan dan filter berdasarkan mata kuliah	Menengah
Tandai favorit	Status favorit bertahan setelah aplikasi ditutup	Menengah
Tiga tabel relasional	Skema relasi mata_kuliah, catatan, kategori/tag	Lanjutan
Ekspor teks	Catatan dapat disalin/dibagikan sesuai kebutuhan	Lanjutan

Fitur lanjutan memerlukan perubahan skema dan pengujian migrasi. Fitur tambahan tidak boleh mengorbankan stabilitas CRUD inti.

6.3 Template backlog PjBL

ID	Kebutuhan pengguna (user story)	Penanggung jawab	Status
US-01	Sebagai mahasiswa, saya ingin menambah catatan agar isi kuliah terdokumentasi.	Ryanda Hermawan	**Selesai**
US-02	Sebagai mahasiswa, saya ingin mengedit catatan agar kesalahan dapat diperbaiki.	Dewi Nofitta Sari	**Selesai**
US-03	Sebagai mahasiswa, saya ingin mencari berdasarkan judul agar mudah ditemukan.	Bintang Sahda Rayyu S.	**Selesai**
US-04	Fitur pilihan kelompok: Deteksi otomatis kuliah aktif ('Catat Sekarang'), skema 3 tabel relasional (jadwal, catatan, kategori) + migrasi v1→v3, urutan terbaru/terlama, favorit permanen, & salin teks.	Seluruh Anggota Tim	**Selesai**

6.4 Siklus proyek

- Definisikan masalah dan kriteria keberhasilan dari sudut pandang pengguna.
- Rancang alur layar dan daftar tugas, bagi peran developer, tester, dan dokumentasi.
- Implementasikan fitur secara incremental dalam branch/commit yang dapat ditelusuri.
- Uji skenario nyata dan catat bug beserta bukti perbaikannya.
- Presentasikan aplikasi langsung dari perangkat Android dan jawab pertanyaan arsitektur.

6.5 Artefak yang harus diserahkan

☑ Tautan repository yang dapat dibaca dosen.

☑ README proyek: instalasi, cara menjalankan, fitur, kontribusi anggota.

☑ Screenshot layar beranda, daftar, form, detail, dan dialog hapus.

☑ Video demo 3–5 menit yang memuat CRUD dan restart aplikasi.

☑ Matriks hasil uji dengan status lulus/gagal dan catatan kendala.

☑ Laporan refleksi individual: apa yang dikerjakan, hambatan, dan pembelajaran.

7. Diagnostik Kesalahan dan Troubleshooting

Gejala	Penyebab umum	Tindakan mahasiswa
flutter tidak dikenali	PATH Flutter belum dikonfigurasi	Periksa lokasi SDK dan ulangi terminal
No devices found	Emulator tidak aktif / USB debugging belum siap	Jalankan emulator, cek flutter devices
Target Chrome terpilih	Salah target platform untuk sqflite Android	Pilih emulator/HP Android
Target of URI does not exist	Dependency belum diunduh / file tidak cocok	flutter pub get, cek impor & struktur file
Undefined name / tipe	Salah ketik atau nama class	Periksa filename, class, import, analisis error
setState called after dispose	Aksi async selesai setelah halaman ditutup	Cek mounted sebelum setState/navigasi
Data hilang sesudah restart	Masih memakai checkpoint M2 / penyimpanan gagal	Cek tahap M3 dan log operasi SQLite
no such table: catatan	Schema belum terbentuk atau DB lama	Cek onCreate/version; uji fresh install dengan hati-hati
Data tidak berubah di daftar	ViewModel tidak notifyListeners / gagal reload	Periksa muatCatatan setelah CRUD
Perubahan skema tak muncul	Database lama dengan versi yang sama	Implementasikan migration; hindari hapus data pengguna

ETIKA DEBUGGING Hindari menghapus data perangkat sebagai solusi pertama. Gunakan cadangan data dummy dan pahami konsekuensi uninstal aplikasi terhadap database.

7.1 Logbook debugging

Tanggal / tahap	Error/observasi	Dugaan	Solusi yang diuji	Hasil
29 Okt 2026 / M3	StateError: databaseFactory not initialized saat widget test	databaseFactory diakses terlalu dini pada konstruktor DatabaseHelper	Ubah ke lazy getter & injeksi FakeCatatanRepository pada widget test	**Teratasi (18/18 tes lulus)**
29 Okt 2026 / Cari	Hasil ketikan lama berisiko menimpa ketikan baru saat cari cepat	Query async SQLite selesai tidak berurutan (race condition)	Tambahkan token urutan permintaan (_permintaan) di CatatanViewModel	**Teratasi (hasil konsisten)**
30 Okt 2026 / DB v3	Penambahan tabel kategori & kolom favorit pada aplikasi terpasang	onCreate tidak dijalankan ulang pada database versi lama	Naikkan versi ke v3 & buat migrasi bertahap di _onUpgrade tanpa hapus data	**Teratasi (migrasi v1→v3 aman)**

8. Penilaian, Umpan Balik, dan Refleksi

8.1 Rubrik PjBL

Aspek	Bobot	Kriteria skor tinggi
Fungsi form, validasi, navigasi	25%	Tidak ada alur macet; umpan balik jelas; navigasi benar
CRUD dan persistensi SQLite	25%	Create/read/update/delete andal; data bertahan setelah restart
Struktur dan kualitas kode	20%	UI, ViewModel, repository, DB terpisah; penamaan jelas
Pengujian dan dokumentasi	15%	Kasus uji beragam; bukti, bug, README lengkap
Demo, refleksi, kontribusi	15%	Mampu menjelaskan keputusan kode dan kontribusi nyata

8.2 Deskriptor kinerja

Level	Rentang	Deskripsi
Sangat baik	85–100	Fitur terpenuhi, pengujian lengkap, kontribusi terbukti, penjelasan teknis tepat
Baik	70–84	Fitur inti berjalan, terdapat beberapa kekurangan minor
Cukup	55–69	Ada fitur inti tidak stabil atau bukti pengujian terbatas
Perlu perbaikan	<55	Bagian utama belum berfungsi atau hasil tidak dapat diverifikasi

8.3 Refleksi individual

Apa perbedaan setState dan ChangeNotifier yang Anda amati dalam praktik?

Jawaban: setState mengelola state lokal di dalam satu StatefulWidget saja (seperti status tombol menyimpan atau teks pencarian), sedangkan ChangeNotifier bersama Provider/Consumer mengelola state bersama lintas layar sehingga penambahan, perubahan, atau penghapusan catatan langsung memperbarui Beranda, Daftar, dan Detail secara reaktif melalui notifyListeners().

Apa fungsi Navigator.push dan Navigator.pop pada alur aplikasi?

Jawaban: Navigator mengelola tumpukan halaman (back stack LIFO). Navigator.push menambahkan route baru di atas tumpukan saat berpindah layar (Beranda → Daftar → Detail → Form), sedangkan Navigator.pop menutup halaman teratas dan mengembalikan pengguna tepat ke halaman sebelumnya sesuai urutan riwayat.

Mengapa ChangeNotifier tidak otomatis membuat data tetap ada setelah restart penuh?

Jawaban: Karena objek ChangeNotifier hanya menyimpan data di memori volatil (RAM) selama proses aplikasi aktif. Ketika pengguna melakukan force stop atau sistem mematikan proses, alokasi RAM dibersihkan. Persistensi permanen membutuhkan penyimpanan non-volatil di perangkat seperti database SQLite (studylog.db) yang dibaca ulang saat aplikasi dibuka kembali.

Apa tanggung jawab DatabaseHelper dibanding Repository dan ViewModel?

Jawaban: DatabaseHelper bertugas mengelola koneksi berkas SQLite, foreign keys, pembuatan tabel (onCreate), seeding, dan migrasi versi (onUpgrade v1→v2→v3). Repository membungkus query SQL berparameter (whereArgs) menjadi operasi objek model Dart (Catatan, Jadwal, Kategori). ViewModel mengelola state tampilan (isLoading, error, filter, urutan) serta memanggil notifyListeners() untuk UI.

Bug paling sulit apa yang ditemukan, bagaimana Anda membuktikan perbaikannya?

Jawaban: Bug paling menantang adalah menjaga migrasi skema dari 1 tabel dasar (v1) ke 3 tabel relasional (v3: jadwal, kategori, catatan) tanpa menghapus catatan lama, serta memisahkan inisialisasi databaseFactory saat widget test. Perbaikannya dibuktikan lewat tes otomatis database_migrasi_test.dart yang menguji upgrade DB v1 ke v3 dan memastikan seluruh data lama tetap utuh tertaut.

Apa kontribusi kode dan pengujian yang paling jelas menunjukkan peran Anda?

Jawaban: Tim kami berkolaborasi utuh: Ryanda Hermawan merancang skema SQLite 3 tabel relasional, fitur deteksi jadwal aktif 'Catat Sekarang', dan repositori Git; Dewi Nofitta Sari membangun alur 5 layar UI, validasi form, filter/urutan, favorit, dan Provider; Bintang Sahda Rayyu Sonora menyusun 18 pengujian otomatis, menguji persistensi di ponsel Android fisik, merekam video demo, dan mendokumentasikan hasil.

9. Lampiran Praktikum dan Referensi

9.1 Nama file yang menjadi acuan

- Checkpoint M1: milestone_01_form/lib/main.dart
- Checkpoint M2: milestone_02_navigation/lib/main.dart
- Checkpoint M3: milestone_03_sqlite/lib/main.dart
- Model: milestone_03_sqlite/lib/models/catatan.dart
- Database: milestone_03_sqlite/lib/data/database_helper.dart
- Repository: milestone_03_sqlite/lib/data/catatan_repository.dart
- ViewModel: milestone_03_sqlite/lib/viewmodels/catatan_view_model.dart
- Layar: milestone_03_sqlite/lib/screens/*.dart
- Tes: milestone_03_sqlite/test/catatan_model_test.dart

9.2 Referensi resmi yang relevan

- Flutter Docs. Persist data with SQLite. https://docs.flutter.dev/cookbook/persistence/sqlite
- Flutter Docs. Navigate to a new screen and back. https://docs.flutter.dev/cookbook/navigation/navigation-basics
- Flutter Docs. Simple app state management (Provider). https://docs.flutter.dev/data-and-backend/state-mgmt/simple
- Flutter Docs. Flutter SDK installation & setup. https://docs.flutter.dev/get-started/install
- Materi internal yang ditelaah: Materi 4 State, Events & User Interaction; Materi 5 Navigation, Lifecycle & ViewModel; Materi 6 Local Persistence & Room Database.

- Modul Mobile Programming yang dilampirkan, khususnya pertemuan 1–9 terkait Flutter UI, form, navigasi, detail data; bagian API pertemuan 10–13 sebagai pengayaan terpisah.

9.3 Catatan untuk dosen / instruktur

Workbook dirancang dengan tiga checkpoint kode agar mahasiswa dapat mereproduksi setiap tahap. Source package memuat file Dart dan pubspec.yaml saja; folder platform Android dihasilkan melalui perintah flutter create. Sebelum dipakai di kelas, dosen harus menjalankan flutter pub get, flutter analyze, flutter test, dan smoke test CRUD menggunakan SDK dan perangkat kelas.

Skor dan alokasi waktu pada dokumen ini merupakan usulan desain pembelajaran, bukan kutipan dari RPS. Perubahan harus diselaraskan dengan CPMK, Sub-CPMK, dan ketentuan penilaian yang berlaku pada program studi.

10. Lampiran Source Code Referensi

Bagian ini memuat kode program sebagaimana disediakan dalam paket checkpoint. Teks program dapat dipakai untuk menelusuri tanggung jawab setiap file. Untuk menyalin kode ke editor, prioritaskan source .dart asli pada paket ZIP agar format tidak berubah ketika disalin dari dokumen Word/PDF.

M1 — Program Form Interaktif

File: milestone_01_form/lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const StudyLogApp());

class StudyLogApp extends StatelessWidget {
  const StudyLogApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'StudyLog | Praktikum 1',
      theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
      home: const CatatanFormSederhana(),
    );
  }
}

class CatatanFormSederhana extends StatefulWidget {
  const CatatanFormSederhana({super.key});

  @override
  State<CatatanFormSederhana> createState() => _CatatanFormSederhanaState();
}

class _CatatanFormSederhanaState extends State<CatatanFormSederhana> {
  final _formKey = GlobalKey<FormState>();
  final _mataKuliah = TextEditingController();
  final _judul = TextEditingController();
  final _isi = TextEditingController();
  String? _hasil;

  @override
  void dispose() {
    _mataKuliah.dispose();
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  void _simpanSementara() {
    if (!_formKey.currentState!.validate()) return;
    setState(() {
      _hasil = '${_mataKuliah.text.trim()} | ${_judul.text.trim()}';
    });
    ScaffoldMessenger.of(context).showSnackBar(
```

```
    const SnackBar(content: Text('Data ditampilkan (belum tersimpan permanen).')),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('StudyLog - Form Catatan')),
    body: ListView(
      padding: const EdgeInsets.all(20),
      children: [
        const Text('Praktikum 1: State & Interaksi',
            style: TextStyle(fontSize: 23, fontWeight: FontWeight.bold)),
        const SizedBox(height: 16),
        Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _mataKuliah,
                decoration: const InputDecoration(labelText: 'Mata kuliah', border:
OutlineInputBorder()),
                validator: (nilai) => (nilai == null || nilai.trim().isEmpty)
                    ? 'Mata kuliah wajib diisi' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _judul,
                decoration: const InputDecoration(labelText: 'Judul catatan', border:
OutlineInputBorder()),
                validator: (nilai) => (nilai == null || nilai.trim().isEmpty)
                    ? 'Judul wajib diisi' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _isi,
                maxLines: 4,
                decoration: const InputDecoration(labelText: 'Isi catatan', border:
OutlineInputBorder()),
                validator: (nilai) => (nilai == null || nilai.trim().isEmpty)
                    ? 'Isi catatan wajib diisi' : null,
              ),
            ],
          ),
        ),
        const SizedBox(height: 18),
        FilledButton.icon(onPressed: _simpanSementara, icon: const Icon(Icons.save),
            label: const Text('Tampilkan Catatan')),
        if (_hasil != null) ...[
          const SizedBox(height: 18),
```

```
            Card(child: Padding(padding: const EdgeInsets.all(16),
              child: Text('Hasil input: $_hasil'))),
          ],
        ],
      ),
    );
  }
}
```

M2 — Navigasi dan State Sementara

File: milestone_02_navigation/lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(ChangeNotifierProvider(
    create: (_) => CatatanStore(),
    child: const StudyLogApp(),
  ));
}

class CatatanMini {
  final String mataKuliah;
  final String judul;
  final String isi;
  const CatatanMini(this.mataKuliah, this.judul, this.isi);
}

class CatatanStore extends ChangeNotifier {
  final List<CatatanMini> _data = [];
  List<CatatanMini> get data => List.unmodifiable(_data);

  void tambah(CatatanMini catatan) {
    _data.add(catatan);
    notifyListeners();
  }
}

class StudyLogApp extends StatelessWidget {
  const StudyLogApp({super.key});
  @override
  Widget build(BuildContext context) => MaterialApp(
        debugShowCheckedModeBanner: false,
        title: 'StudyLog | Praktikum 2',
        theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
        home: const BerandaPage(),
      );
}

class BerandaPage extends StatelessWidget {
  const BerandaPage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(title: const Text('StudyLog')),
        body: Center(child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
```

```
          const Icon(Icons.menu_book_rounded, size: 72),
          const SizedBox(height: 12),
          const Text('Catatan perkuliahan', style: TextStyle(fontSize: 24)),
          const SizedBox(height: 20),
          FilledButton(onPressed: () => Navigator.push(
            context, MaterialPageRoute(builder: (_) => const DaftarPage())),
            child: const Text('Buka Daftar Catatan')),
        ],
      )),
    );
}

class DaftarPage extends StatelessWidget {
  const DaftarPage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text('Daftar Catatan')),
      body: Consumer<CatatanStore>(builder: (context, store, _) {
        if (store.data.isEmpty) {
          return const Center(child: Text('Belum ada catatan. Tambahkan melalui tombol +.'));
        }
        return ListView.builder(
          itemCount: store.data.length,
          itemBuilder: (context, index) {
            final item = store.data[index];
            return ListTile(
              title: Text(item.judul),
              subtitle: Text(item.mataKuliah),
              trailing: const Icon(Icons.chevron_right),
              onTap: () => Navigator.push(context,
                MaterialPageRoute(builder: (_) => DetailPage(catatan: item))),
            );
          },
        );
      }),
      floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.push(context,
          MaterialPageRoute(builder: (_) => const FormPage())),
        child: const Icon(Icons.add),
      ),
    );
}

class DetailPage extends StatelessWidget {
  final CatatanMini catatan;
  const DetailPage({super.key, required this.catatan});
  @override
  Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text('Detail Catatan')),
      body: Padding(padding: const EdgeInsets.all(20), child: Column(
```

```
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(catatan.mataKuliah, style: Theme.of(context).textTheme.labelLarge),
          const SizedBox(height: 12),
          Text(catatan.judul, style: Theme.of(context).textTheme.headlineMedium),
          const Divider(height: 32),
          Text(catatan.isi),
        ],
      )),
    );
}

class FormPage extends StatefulWidget {
 const FormPage({super.key});
 @override
 State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {
 final _form = GlobalKey<FormState>();
 final _mk = TextEditingController();
 final _judul = TextEditingController();
 final _isi = TextEditingController();

 @override
 void dispose() {
   _mk.dispose(); _judul.dispose(); _isi.dispose();
  super.dispose();
 }

 @override
 Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text('Tambah Catatan')),
      body: ListView(padding: const EdgeInsets.all(20), children: [
        Form(key: _form, child: Column(children: [
          _field(_mk, 'Mata kuliah'),
          const SizedBox(height: 12),
          _field(_judul, 'Judul'),
          const SizedBox(height: 12),
          _field(_isi, 'Isi', baris: 4),
        ])),
        const SizedBox(height: 20),
        FilledButton(onPressed: () {
          if (!_form.currentState!.validate()) return;
          context.read<CatatanStore>().tambah(CatatanMini(
            _mk.text.trim(), _judul.text.trim(), _isi.text.trim()));
          Navigator.pop(context);
        }, child: const Text('Simpan Sementara')),
      ]),
    );
```

```
  Widget _field(TextEditingController controller, String label, {int baris = 1}) {
    return TextFormField(
      controller: controller, maxLines: baris,
      decoration: InputDecoration(labelText: label, border: const OutlineInputBorder()),
      validator: (nilai) => (nilai == null || nilai.trim().isEmpty)
          ? '$label wajib diisi' : null,
    );
  }
}
```

M3 — Model Data Catatan

File: milestone_03_sqlite/lib/models/catatan.dart

```
class Catatan {
  final int? id;
  final String mataKuliah;
  final String judul;
  final String isi;
  final DateTime tanggal;

  const Catatan({
    this.id,
    required this.mataKuliah,
    required this.judul,
    required this.isi,
    required this.tanggal,
  });

  Map<String, Object?> toMap() => {
        if (id != null) 'id': id,
        'mata_kuliah': mataKuliah,
        'judul': judul,
        'isi': isi,
        'tanggal': tanggal.toIso8601String(),
      };

  factory Catatan.fromMap(Map<String, Object?> data) => Catatan(
        id: data['id'] as int?,
        mataKuliah: data['mata_kuliah'] as String,
        judul: data['judul'] as String,
        isi: data['isi'] as String,
        tanggal: DateTime.parse(data['tanggal'] as String),
      );
}
```

M3 — DatabaseHelper SQLite

File: milestone_03_sqlite/lib/data/database_helper.dart

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class DatabaseHelper {
  DatabaseHelper._();
  static final DatabaseHelper instance = DatabaseHelper._();
  Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    final filePath = join(await getDatabasesPath(), 'studylog.db');
    _database = await openDatabase(
      filePath,
      version: 1,
      onCreate: (db, version) async {
        await db.execute("""
          CREATE TABLE catatan (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            mata_kuliah TEXT NOT NULL,
            judul TEXT NOT NULL,
            isi TEXT NOT NULL,
            tanggal TEXT NOT NULL
          )
        """);
      },
    );
    return _database!;
  }
}
```

M3 — Repository Catatan

File: milestone_03_sqlite/lib/data/catatan_repository.dart

```
import 'package:sqflite/sqflite.dart';

import '../models/catatan.dart';
import 'database_helper.dart';

class CatatanRepository {
  final DatabaseHelper helper;
  CatatanRepository(this.helper);

  Future<List<Catatan>> semua() async {
    final Database db = await helper.database;
    final baris = await db.query('catatan', orderBy: 'tanggal DESC, id DESC');
    return baris.map(Catatan.fromMap).toList();
  }

  Future<int> tambah(Catatan catatan) async {
    final db = await helper.database;
    return db.insert('catatan', catatan.toMap());
  }

  Future<int> ubah(Catatan catatan) async {
    if (catatan.id == null) throw ArgumentError('ID catatan tidak boleh kosong');
    final db = await helper.database;
    return db.update('catatan', catatan.toMap(),
        where: 'id = ?', whereArgs: [catatan.id]);
  }

  Future<int> hapus(int id) async {
    final db = await helper.database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}
```

M3 — ViewModel ChangeNotifier

File: milestone_03_sqlite/lib/viewmodels/catatan_view_model.dart

```
import 'package:flutter/foundation.dart';

import '../data/catatan_repository.dart';
import '../models/catatan.dart';

class CatatanViewModel extends ChangeNotifier {
  final CatatanRepository repository;
  CatatanViewModel(this.repository);

  List<Catatan> _catatan = [];
  List<Catatan> get catatan => List.unmodifiable(_catatan);
  bool isLoading = false;
  String? error;

  Future<void> muatCatatan() async {
    isLoading = true;
    error = null;
    notifyListeners();
    try {
      _catatan = await repository.semua();
    } catch (e) {
      error = 'Tidak dapat membaca catatan: $e';
    } finally {
      isLoading = false;
      notifyListeners();
    }
  }

  Future<void> tambahCatatan(Catatan baru) async {
    await repository.tambah(baru);
    await muatCatatan();
  }

  Future<void> ubahCatatan(Catatan catatan) async {
    await repository.ubah(catatan);
    await muatCatatan();
  }

  Future<void> hapusCatatan(int id) async {
    await repository.hapus(id);
    await muatCatatan();
  }
}
```

M3 — Entry Point Aplikasi

File: milestone_03_sqlite/lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'data/catatan_repository.dart';
import 'data/database_helper.dart';
import 'screens/beranda_page.dart';
import 'viewmodels/catatan_view_model.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(
    ChangeNotifierProvider(
      create: (_) => CatatanViewModel(
        CatatanRepository(DatabaseHelper.instance),
      )..muatCatatan(),
      child: const StudyLogApp(),
    ),
  );
}

class StudyLogApp extends StatelessWidget {
  const StudyLogApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
        title: 'StudyLog',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorSchemeSeed: const Color(0xFF4158D0),
          useMaterial3: true,
          inputDecorationTheme: const InputDecorationTheme(
            border: OutlineInputBorder(),
          ),
        ),
        home: const BerandaPage(),
      );
}
```

M3 — Halaman Beranda

File: milestone_03_sqlite/lib/screens/beranda_page.dart

```
import 'package:flutter/material.dart';
import 'daftar_catatan_page.dart';

class BerandaPage extends StatelessWidget {
  const BerandaPage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(title: const Text('StudyLog')),
        body: Center(child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(mainAxisSize: MainAxisSize.min, children: [
            Icon(Icons.menu_book_rounded, size: 84,
                color: Theme.of(context).colorScheme.primary),
            const SizedBox(height: 24),
            Text('Kelola catatan kuliahmu',
                style: Theme.of(context).textTheme.headlineSmall,
                textAlign: TextAlign.center),
            const SizedBox(height: 12),
            const Text('Simpan catatan di perangkat. Tetap dapat dibuka tanpa internet.',
                textAlign: TextAlign.center),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: () => Navigator.push(context,
                  MaterialPageRoute(builder: (_) => const DaftarCatatanPage())),
              icon: const Icon(Icons.arrow_forward),
              label: const Text('Lihat Catatan'),
            ),
          ]),
        )),
      );
}
```

M3 — Halaman Daftar Catatan

File: milestone_03_sqlite/lib/screens/daftar_catatan_page.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../viewmodels/catatan_view_model.dart';
import 'detail_catatan_page.dart';
import 'form_catatan_page.dart';

class DaftarCatatanPage extends StatelessWidget {
 const DaftarCatatanPage({super.key});

 @override
 Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text('Daftar Catatan')),
      body: Consumer<CatatanViewModel>(builder: (context, vm, _) {
       if (vm.isLoading) {
        return const Center(child: CircularProgressIndicator());
       }
       if (vm.error != null) {
        return Center(child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
           Text(vm.error!, textAlign: TextAlign.center),
           TextButton(onPressed: vm.muatCatatan, child: const Text('Coba lagi')),
          ],
        ));
       }
       if (vm.catatan.isEmpty) {
        return const Center(child: Padding(
          padding: EdgeInsets.all(24),
          child: Text('Belum ada catatan. Tekan + untuk menambah catatan.',
             textAlign: TextAlign.center),
        ));
       }
       return ListView.separated(
        padding: const EdgeInsets.all(12),
        itemCount: vm.catatan.length,
        separatorBuilder: (_, __) => const SizedBox(height: 8),
        itemBuilder: (context, index) {
         final catatan = vm.catatan[index];
         return Card(child: ListTile(
          title: Text(catatan.judul),
          subtitle: Text(catatan.mataKuliah),
          trailing: const Icon(Icons.chevron_right),
          onTap: () => Navigator.push(context, MaterialPageRoute(
           builder: (_) => DetailCatatanPage(id: catatan.id!),
          )),
```

```
            ));
          },
        );
      }),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => Navigator.push(context,
          MaterialPageRoute(builder: (_) => const FormCatatanPage())),
        icon: const Icon(Icons.add),
        label: const Text('Tambah'),
      ),
    );
}
```

M3 — Halaman Form Tambah/Ubah

File: milestone_03_sqlite/lib/screens/form_catatan_page.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../viewmodels/catatan_view_model.dart';

class FormCatatanPage extends StatefulWidget {
  final Catatan? awal;
  const FormCatatanPage({super.key, this.awal});

  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _mataKuliah;
  late final TextEditingController _judul;
  late final TextEditingController _isi;
  bool _menyimpan = false;

  @override
  void initState() {
    super.initState();
    _mataKuliah = TextEditingController(text: widget.awal?.mataKuliah ?? '');
    _judul = TextEditingController(text: widget.awal?.judul ?? '');
    _isi = TextEditingController(text: widget.awal?.isi ?? '');
  }

  @override
  void dispose() {
    _mataKuliah.dispose();
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate() || _menyimpan) return;
    setState(() => _menyimpan = true);
    final data = Catatan(
      id: widget.awal?.id,
      mataKuliah: _mataKuliah.text.trim(),
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
      tanggal: widget.awal?.tanggal ?? DateTime.now(),
```

```
    );
    try {
      final vm = context.read<CatatanViewModel>();
      if (widget.awal == null) {
        await vm.tambahCatatan(data);
      } else {
        await vm.ubahCatatan(data);
      }
      if (!mounted) return;
      Navigator.pop(context, true);
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Gagal menyimpan: $e')));
    } finally {
      if (mounted) setState(() => _menyimpan = false);
    }
  }

  Widget _input(TextEditingController kontrol, String label, {int baris = 1}) {
    return TextFormField(
      controller: kontrol,
      maxLines: baris,
      decoration: InputDecoration(labelText: label),
      validator: (teks) => (teks == null || teks.trim().isEmpty)
          ? '$label wajib diisi' : null,
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
        appBar: AppBar(title: Text(widget.awal == null
            ? 'Tambah Catatan' : 'Ubah Catatan')),
        body: ListView(padding: const EdgeInsets.all(20), children: [
          Form(key: _formKey, child: Column(children: [
            _input(_mataKuliah, 'Mata kuliah'),
            const SizedBox(height: 14),
            _input(_judul, 'Judul'),
            const SizedBox(height: 14),
            _input(_isi, 'Isi catatan', baris: 5),
          ])),
          const SizedBox(height: 22),
          FilledButton.icon(
            onPressed: _menyimpan ? null : _simpan,
            icon: const Icon(Icons.save),
            label: Text(_menyimpan ? 'Menyimpan...' : 'Simpan'),
          ),
        ]),
      );
}
```

M3 — Halaman Detail & Hapus

File: milestone_03_sqlite/lib/screens/detail_catatan_page.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../viewmodels/catatan_view_model.dart';
import 'form_catatan_page.dart';

class DetailCatatanPage extends StatelessWidget {
 final int id;
 const DetailCatatanPage({super.key, required this.id});

 @override
 Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text('Detail Catatan')),
      body: Consumer<CatatanViewModel>(builder: (context, vm, _) {
       Catatan? catatan;
       for (final item in vm.catatan) {
        if (item.id == id) {
         catatan = item;
         break;
        }
       }
       if (catatan == null) {
        return const Center(child: Text('Catatan tidak ditemukan.'));
       }
       final data = catatan;
       return ListView(padding: const EdgeInsets.all(20), children: [
        Text(data.mataKuliah, style: Theme.of(context).textTheme.titleMedium),
        const SizedBox(height: 8),
        Text(data.judul, style: Theme.of(context).textTheme.headlineSmall),
        const SizedBox(height: 8),
        Text('Dibuat: ${data.tanggal.day}/${data.tanggal.month}/${data.tanggal.year}'),
        const Divider(height: 32),
        Text(data.isi),
        const SizedBox(height: 28),
        FilledButton.tonalIcon(
         onPressed: () => Navigator.push(context,
             MaterialPageRoute(builder: (_) => FormCatatanPage(awal: data))),
         icon: const Icon(Icons.edit), label: const Text('Ubah'),
        ),
        const SizedBox(height: 10),
        OutlinedButton.icon(
         onPressed: () => _konfirmasiHapus(context, vm, data.id!),
         icon: const Icon(Icons.delete_outline), label: const Text('Hapus'),
        ),
       ]);
```

```
      }),
    );

Future<void> _konfirmasiHapus(
    BuildContext context, CatatanViewModel vm, int id) async {
  final yakin = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Hapus catatan?'),
      content: const Text('Catatan yang dihapus tidak dapat dikembalikan.'),
      actions: [
        TextButton(onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal')),
        FilledButton(onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus')),
      ],
    ),
  );
  if (yakin != true || !context.mounted) return;
  try {
    await vm.hapusCatatan(id);
    if (context.mounted) Navigator.pop(context);
  } catch (e) {
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Gagal menghapus: $e')));
  }
 }
}
```

M3 — Pengujian Model Data

File: milestone_03_sqlite/test/catatan_model_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:studylog_flutter/models/catatan.dart';

void main() {
  test('Catatan round-trip Map tidak mengubah data', () {
    final asli = Catatan(
      id: 10, mataKuliah: 'Pemrograman Mobile',
      judul: 'SQLite', isi: 'Materi CRUD',
      tanggal: DateTime(2026, 10, 22),
    );
    final hasil = Catatan.fromMap(asli.toMap());
    expect(hasil.id, 10);
    expect(hasil.judul, 'SQLite');
    expect(hasil.mataKuliah, 'Pemrograman Mobile');
    expect(hasil.isi, 'Materi CRUD');
  });
}
```

Akhir lampiran source code. Lanjutkan dengan menguji kode pada SDK Flutter dan perangkat Android yang tersedia.